Data Transformer

Advanced SQL Project – Query Results

Generated on 02-Nov-2026 | Engine: SQLite 3.45.1 | Queries executed: 17

Contents

Q1. INNER JOIN – Retrieve all orders and customer details where orders exist.

Q2. LEFT JOIN – Retrieve all customers and their corresponding orders (if any).

Q3. RIGHT JOIN – Retrieve all orders and their corresponding customers (if any).

Q4. FULL OUTER JOIN – Retrieve all customers and all orders, regardless of matching.

Q5. Subquery (customers) – Find customers who have placed orders worth more than the average amount.

Q6. Subquery (employees) – Find employees with salaries above the average salary.

Q7. Date extraction – Extract the year and month from the OrderDate.

Q8. Date difference – Calculate the difference in days between two dates (order date and current date).

Q9. Date formatting – Format the OrderDate to a more readable format (e.g., 'DD-MMM-YYYY').

Q10. String concatenation – Concatenate FirstName and LastName to form a full name.

Q11. REPLACE – Replace part of a string (e.g., replace 'John' with 'Jonathan').

Q12. UPPER / LOWER – Convert FirstName to uppercase and LastName to lowercase.

Q13. TRIM – Trim extra spaces from the Email field.

Q14. Window: running total – Calculate the running total of TotalAmount for each order.

Q15. Window: RANK() – Rank orders based on TotalAmount using the RANK() function.

Q16. CASE: discount – Assign a discount based on TotalAmount (>1000: 10% off, >500: 5% off).

Q17. CASE: salary bands – Categorize employees' salaries as low, medium, or high (Low < 45,000 | Medium 45,000-89,999 | High >= 90,000).

Source Data (Sample Tables)

Customers (10 rows)

CustomerID	FirstName	LastName	Email	RegistrationDate
1	John	Doe	john.doe@email.com	2022-03-15
2	Jane	Smith	jane.smith@email.com	2021-11-02
3	Johnny	Appleseed	johnny.a@email.com	2022-06-20
4	Priya	Patel	priya.patel@email.com	2023-01-10
5	Rahul	Mehta	rahul.mehta@email.com	2022-09-05
6	Emily	Johnson	emily.j@email.com	2023-02-14
7	Carlos	Gomez	carlos.g@email.com	2021-05-30
8	Aisha	Khan	aisha.khan@email.com	2023-04-01
9	John	Carter	john.carter@email.com	2022-12-12
10	Mei	Lin	mei.lin@email.com	2023-05-18

Orders (12 rows)

OrderID	CustomerID	OrderDate	TotalAmount
101	1	2023-07-01	150.50
102	2	2023-07-03	200.75
103	3	2023-07-15	1,250.00
104	4	2023-08-02	620.40
105	1	2023-08-10	75.25
106	5	2023-08-21	980.00
107	6	2023-09-05	310.00
108	2	2023-09-18	1,520.90
109	4	2023-10-01	45.99
110	8	2023-10-12	500.00
111	NULL	2023-10-20	89.99
112	9	2023-11-03	275.00

Employees (8 rows)

EmployeeID	FirstName	LastName	Department	HireDate	Salary
1	Mark	Johnson	Sales	2020-01-15	50,000.00
2	Susan	Lee	HR	2021-03-20	55,000.00
3	Anil	Sharma	IT	2019-07-01	92,000.00
4	Laura	Chen	Finance	2022-05-16	78,000.00
5	David	Brown	Sales	2018-11-12	35,000.00
6	Neha	Verma	Marketing	2023-02-01	42,000.00
7	Robert	King	IT	2017-09-25	115,000.00
8	Fatima	Noor	HR	2022-10-10	38,000.00

Query 1: INNER JOIN

Task: Retrieve all orders and customer details where orders exist.

```
SELECT o.OrderID, o.OrderDate, o.TotalAmount,
       c.CustomerID, c.FirstName, c.LastName
FROM   Orders o
INNER JOIN Customers c ON c.CustomerID = o.CustomerID
ORDER BY o.OrderID;
```

Result – 11 row(s)

OrderID	OrderDate	TotalAmount	CustomerID	FirstName	LastName
101	2023-07-01	150.50	1	John	Doe
102	2023-07-03	200.75	2	Jane	Smith
103	2023-07-15	1,250.00	3	Johnny	Appleseed
104	2023-08-02	620.40	4	Priya	Patel
105	2023-08-10	75.25	1	John	Doe
106	2023-08-21	980.00	5	Rahul	Mehta
107	2023-09-05	310.00	6	Emily	Johnson
108	2023-09-18	1,520.90	2	Jane	Smith
109	2023-10-01	45.99	4	Priya	Patel
110	2023-10-12	500.00	8	Aisha	Khan
112	2023-11-03	275.00	9	John	Carter

Query 2: LEFT JOIN

Task: Retrieve all customers and their corresponding orders (if any).

```
SELECT c.CustomerID, c.FirstName, c.LastName,
       o.OrderID, o.OrderDate, o.TotalAmount
FROM   Customers c
LEFT JOIN Orders o ON o.CustomerID = c.CustomerID
ORDER BY c.CustomerID, o.OrderID;
```

Result – 13 row(s)

CustomerID	FirstName	LastName	OrderID	OrderDate	TotalAmount
1	John	Doe	101	2023-07-01	150.50
1	John	Doe	105	2023-08-10	75.25
2	Jane	Smith	102	2023-07-03	200.75
2	Jane	Smith	108	2023-09-18	1,520.90
3	Johnny	Appleseed	103	2023-07-15	1,250.00
4	Priya	Patel	104	2023-08-02	620.40
4	Priya	Patel	109	2023-10-01	45.99
5	Rahul	Mehta	106	2023-08-21	980.00
6	Emily	Johnson	107	2023-09-05	310.00
7	Carlos	Gomez	NULL	NULL	NULL
8	Aisha	Khan	110	2023-10-12	500.00
9	John	Carter	112	2023-11-03	275.00
10	Mei	Lin	NULL	NULL	NULL

Query 3: RIGHT JOIN

Task: Retrieve all orders and their corresponding customers (if any).

```
SELECT o.OrderID, o.OrderDate, o.TotalAmount,
       c.CustomerID, c.FirstName, c.LastName
FROM   Customers c
RIGHT JOIN Orders o ON o.CustomerID = c.CustomerID
ORDER BY o.OrderID;
```

Result – 12 row(s)

OrderID	OrderDate	TotalAmount	CustomerID	FirstName	LastName
101	2023-07-01	150.50	1	John	Doe
102	2023-07-03	200.75	2	Jane	Smith
103	2023-07-15	1,250.00	3	Johnny	Appleseed
104	2023-08-02	620.40	4	Priya	Patel
105	2023-08-10	75.25	1	John	Doe
106	2023-08-21	980.00	5	Rahul	Mehta
107	2023-09-05	310.00	6	Emily	Johnson
108	2023-09-18	1,520.90	2	Jane	Smith
109	2023-10-01	45.99	4	Priya	Patel
110	2023-10-12	500.00	8	Aisha	Khan
111	2023-10-20	89.99	NULL	NULL	NULL
112	2023-11-03	275.00	9	John	Carter

Query 4: FULL OUTER JOIN

Task: Retrieve all customers and all orders, regardless of matching.

```
SELECT c.CustomerID, c.FirstName, c.LastName,
       o.OrderID, o.OrderDate, o.TotalAmount
FROM   Customers c
FULL OUTER JOIN Orders o ON o.CustomerID = c.CustomerID
ORDER BY c.CustomerID, o.OrderID;
```

Result – 14 row(s)

CustomerID	FirstName	LastName	OrderID	OrderDate	TotalAmount
NULL	NULL	NULL	111	2023-10-20	89.99
1	John	Doe	101	2023-07-01	150.50
1	John	Doe	105	2023-08-10	75.25
2	Jane	Smith	102	2023-07-03	200.75
2	Jane	Smith	108	2023-09-18	1,520.90
3	Johnny	Appleseed	103	2023-07-15	1,250.00
4	Priya	Patel	104	2023-08-02	620.40
4	Priya	Patel	109	2023-10-01	45.99
5	Rahul	Mehta	106	2023-08-21	980.00
6	Emily	Johnson	107	2023-09-05	310.00
7	Carlos	Gomez	NULL	NULL	NULL
8	Aisha	Khan	110	2023-10-12	500.00
9	John	Carter	112	2023-11-03	275.00
10	Mei	Lin	NULL	NULL	NULL

Query 5: Subquery (customers)

Task: Find customers who have placed orders worth more than the average amount.

```
SELECT CustomerID, FirstName, LastName, Email
FROM   Customers
WHERE  CustomerID IN (
           SELECT CustomerID
           FROM   Orders
           WHERE  TotalAmount > (SELECT AVG(TotalAmount) FROM Orders)
       )
ORDER BY CustomerID;
```

Result – 4 row(s)

CustomerID	FirstName	LastName	Email
2	Jane	Smith	jane.smith@email.com
3	Johnny	Appleseed	johnny.a@email.com
4	Priya	Patel	priya.patel@email.com
5	Rahul	Mehta	rahul.mehta@email.com

Query 6: Subquery (employees)

Task: Find employees with salaries above the average salary.

```
SELECT EmployeeID, FirstName, LastName, Department, Salary
FROM   Employees
WHERE  Salary > (SELECT AVG(Salary) FROM Employees)
ORDER BY Salary DESC;
```

Result – 3 row(s)

EmployeeID	FirstName	LastName	Department	Salary
7	Robert	King	IT	115,000.00
3	Anil	Sharma	IT	92,000.00
4	Laura	Chen	Finance	78,000.00

Query 7: Date extraction

Task: Extract the year and month from the OrderDate.

```
SELECT OrderID, OrderDate,
       CAST(strftime('%Y', OrderDate) AS INTEGER) AS OrderYear,
       CAST(strftime('%m', OrderDate) AS INTEGER) AS OrderMonth
FROM   Orders
ORDER BY OrderID;
```

Result – 12 row(s)

OrderID	OrderDate	OrderYear	OrderMonth
101	2023-07-01	2023	7
102	2023-07-03	2023	7
103	2023-07-15	2023	7
104	2023-08-02	2023	8
105	2023-08-10	2023	8
106	2023-08-21	2023	8
107	2023-09-05	2023	9
108	2023-09-18	2023	9
109	2023-10-01	2023	10
110	2023-10-12	2023	10
111	2023-10-20	2023	10
112	2023-11-03	2023	11

Query 8: Date difference

Task: Calculate the difference in days between two dates (order date and current date).

```
SELECT OrderID, OrderDate, date('now') AS CurrentDate,
       CAST(julianday(date('now')) - julianday(OrderDate) AS INTEGER) AS DaysSinceOrder
FROM   Orders
ORDER BY OrderID;
```

Result – 12 row(s)

OrderID	OrderDate	CurrentDate	DaysSinceOrder
101	2023-07-01	2026-11-02	1220
102	2023-07-03	2026-11-02	1218
103	2023-07-15	2026-11-02	1206
104	2023-08-02	2026-11-02	1188
105	2023-08-10	2026-11-02	1180
106	2023-08-21	2026-11-02	1169
107	2023-09-05	2026-11-02	1154
108	2023-09-18	2026-11-02	1141
109	2023-10-01	2026-11-02	1128
110	2023-10-12	2026-11-02	1117
111	2023-10-20	2026-11-02	1109
112	2023-11-03	2026-11-02	1095

Query 9: Date formatting

Task: Format the OrderDate to a more readable format (e.g., 'DD-MMM-YYYY').

```
SELECT OrderID, OrderDate,
       strftime('%d', OrderDate) || '-' ||
       substr('JanFebMarAprMayJunJulAugSepOctNovDec',
              (CAST(strftime('%m', OrderDate) AS INTEGER) - 1) * 3 + 1, 3)
       || '-' || strftime('%Y', OrderDate) AS FormattedOrderDate
FROM   Orders
ORDER BY OrderID;
```

Result – 12 row(s)

OrderID	OrderDate	FormattedOrderDate
101	2023-07-01	01-Jul-2023
102	2023-07-03	03-Jul-2023
103	2023-07-15	15-Jul-2023
104	2023-08-02	02-Aug-2023
105	2023-08-10	10-Aug-2023
106	2023-08-21	21-Aug-2023
107	2023-09-05	05-Sep-2023
108	2023-09-18	18-Sep-2023
109	2023-10-01	01-Oct-2023
110	2023-10-12	12-Oct-2023
111	2023-10-20	20-Oct-2023
112	2023-11-03	03-Nov-2023

Query 10: String concatenation

Task: Concatenate FirstName and LastName to form a full name.

```
SELECT CustomerID, FirstName, LastName,
       FirstName || ' ' || LastName AS FullName
FROM   Customers
ORDER BY CustomerID;
```

Result – 10 row(s)

CustomerID	FirstName	LastName	FullName
1	John	Doe	John Doe
2	Jane	Smith	Jane Smith
3	Johnny	Appleseed	Johnny Appleseed
4	Priya	Patel	Priya Patel
5	Rahul	Mehta	Rahul Mehta
6	Emily	Johnson	Emily Johnson
7	Carlos	Gomez	Carlos Gomez
8	Aisha	Khan	Aisha Khan
9	John	Carter	John Carter
10	Mei	Lin	Mei Lin

Query 11: REPLACE

Task: Replace part of a string (e.g., replace 'John' with 'Jonathan').

```
SELECT CustomerID,
       FirstName || ' ' || LastName AS FullName,
       REPLACE(FirstName || ' ' || LastName, 'John', 'Jonathan') AS ReplacedName
FROM   Customers
ORDER BY CustomerID;
```

Result – 10 row(s)

CustomerID	FullName	ReplacedName
1	John Doe	Jonathan Doe
2	Jane Smith	Jane Smith
3	Johnny Appleseed	Jonathanny Appleseed
4	Priya Patel	Priya Patel
5	Rahul Mehta	Rahul Mehta
6	Emily Johnson	Emily Jonathanson
7	Carlos Gomez	Carlos Gomez
8	Aisha Khan	Aisha Khan
9	John Carter	Jonathan Carter
10	Mei Lin	Mei Lin

Query 12: UPPER / LOWER

Task: Convert FirstName to uppercase and LastName to lowercase.

```
SELECT CustomerID, FirstName, LastName,
       UPPER(FirstName) AS FirstNameUpper,
       LOWER(LastName)  AS LastNameLower
FROM   Customers
ORDER BY CustomerID;
```

Result – 10 row(s)

CustomerID	FirstName	LastName	FirstNameUpper	LastNameLower
1	John	Doe	JOHN	doe
2	Jane	Smith	JANE	smith
3	Johnny	Appleseed	JOHNNY	appleseed
4	Priya	Patel	PRIYA	patel
5	Rahul	Mehta	RAHUL	mehta
6	Emily	Johnson	EMILY	johnson
7	Carlos	Gomez	CARLOS	gomez
8	Aisha	Khan	AISHA	khan
9	John	Carter	JOHN	carter
10	Mei	Lin	MEI	lin

Query 13: TRIM

Task: Trim extra spaces from the Email field.

```
SELECT CustomerID,
       '[' || Email || ']'       AS EmailBefore,
       '[' || TRIM(Email) || ']' AS EmailAfter,
       LENGTH(Email) - LENGTH(TRIM(Email)) AS SpacesRemoved
FROM   Customers
ORDER BY CustomerID;
```

Result – 10 row(s)

CustomerID	EmailBefore	EmailAfter	SpacesRemoved
1	[john.doe@email.com]	[john.doe@email.com]	0
2	[jane.smith@email.com]	[jane.smith@email.com]	0
3	[johnny.a@email.com]	[johnny.a@email.com]	4
4	[priya.patel@email.com]	[priya.patel@email.com]	0
5	[rahul.mehta@email.com]	[rahul.mehta@email.com]	2
6	[emily.j@email.com]	[emily.j@email.com]	0
7	[carlos.g@email.com]	[carlos.g@email.com]	0
8	[aisha.khan@email.com]	[aisha.khan@email.com]	3
9	[john.carter@email.com]	[john.carter@email.com]	2
10	[mei.lin@email.com]	[mei.lin@email.com]	0

Query 14: Window: running total

Task: Calculate the running total of TotalAmount for each order.

```
SELECT OrderID, OrderDate, TotalAmount,
       SUM(TotalAmount) OVER (ORDER BY OrderDate, OrderID
                              ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS RunningTotal
FROM   Orders
ORDER BY OrderDate, OrderID;
```

Result – 12 row(s)

OrderID	OrderDate	TotalAmount	RunningTotal
101	2023-07-01	150.50	150.50
102	2023-07-03	200.75	351.25
103	2023-07-15	1,250.00	1,601.25
104	2023-08-02	620.40	2,221.65
105	2023-08-10	75.25	2,296.90
106	2023-08-21	980.00	3,276.90
107	2023-09-05	310.00	3,586.90
108	2023-09-18	1,520.90	5,107.80
109	2023-10-01	45.99	5,153.79
110	2023-10-12	500.00	5,653.79
111	2023-10-20	89.99	5,743.78
112	2023-11-03	275.00	6,018.78

Query 15: Window: RANK()

Task: Rank orders based on TotalAmount using the RANK() function.

```
SELECT OrderID, OrderDate, TotalAmount,
       RANK() OVER (ORDER BY TotalAmount DESC) AS AmountRank
FROM   Orders
ORDER BY AmountRank, OrderID;
```

Result – 12 row(s)

OrderID	OrderDate	TotalAmount	AmountRank
108	2023-09-18	1,520.90	1
103	2023-07-15	1,250.00	2
106	2023-08-21	980.00	3
104	2023-08-02	620.40	4
110	2023-10-12	500.00	5
107	2023-09-05	310.00	6
112	2023-11-03	275.00	7
102	2023-07-03	200.75	8
101	2023-07-01	150.50	9
111	2023-10-20	89.99	10
105	2023-08-10	75.25	11
109	2023-10-01	45.99	12

Query 16: CASE: discount

Task: Assign a discount based on TotalAmount (>1000: 10% off, >500: 5% off).

```
SELECT OrderID, TotalAmount,
       CASE WHEN TotalAmount > 1000 THEN 10
            WHEN TotalAmount > 500  THEN 5
            ELSE 0 END AS DiscountPct,
       ROUND(TotalAmount *
             CASE WHEN TotalAmount > 1000 THEN 0.10
                  WHEN TotalAmount > 500  THEN 0.05
                  ELSE 0 END, 2) AS DiscountAmount,
       ROUND(TotalAmount *
             (1 - CASE WHEN TotalAmount > 1000 THEN 0.10
                       WHEN TotalAmount > 500  THEN 0.05
                       ELSE 0 END), 2) AS FinalAmount
FROM   Orders
ORDER BY OrderID;
```

Result – 12 row(s)

OrderID	TotalAmount	DiscountPct	DiscountAmount	FinalAmount
101	150.50	0	0.00	150.50
102	200.75	0	0.00	200.75
103	1,250.00	10	125.00	1,125.00
104	620.40	5	31.02	589.38
105	75.25	0	0.00	75.25
106	980.00	5	49.00	931.00
107	310.00	0	0.00	310.00
108	1,520.90	10	152.09	1,368.81
109	45.99	0	0.00	45.99
110	500.00	0	0.00	500.00
111	89.99	0	0.00	89.99
112	275.00	0	0.00	275.00

Query 17: CASE: salary bands

Task: Categorize employees' salaries as low, medium, or high (Low < 45,000 | Medium 45,000-89,999 | High >= 90,000).

```
SELECT EmployeeID, FirstName, LastName, Department, Salary,
       CASE WHEN Salary < 45000 THEN 'Low'
            WHEN Salary < 90000 THEN 'Medium'
            ELSE 'High' END AS SalaryCategory
FROM   Employees
ORDER BY Salary;
```

Result – 8 row(s)

EmployeeID	FirstName	LastName	Department	Salary	SalaryCategory
5	David	Brown	Sales	35,000.00	Low
8	Fatima	Noor	HR	38,000.00	Low
6	Neha	Verma	Marketing	42,000.00	Low
1	Mark	Johnson	Sales	50,000.00	Medium
2	Susan	Lee	HR	55,000.00	Medium
4	Laura	Chen	Finance	78,000.00	Medium
3	Anil	Sharma	IT	92,000.00	High
7	Robert	King	IT	115,000.00	High